

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 3

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 13, 2026

Part I

NUMPY: LINEAR ALGEBRA AND NUMERICAL ANALYSIS

WHY LINEAR ALGEBRA IS CENTRAL TO SCIENTIFIC COMPUTING

Many scientific problems reduce to linear algebra.

Examples include:

- ▶ solving systems of equations,
- ▶ fitting models to data,
- ▶ simulating physical systems,
- ▶ machine learning and optimization.

NumPy provides efficient tools to express and solve these problems directly in code.

Key idea:

Scientific computing is linear algebra executed at scale.

VECTORS AND MATRICES IN NUMPY

In NumPy:

- ▶ vectors are 1D arrays,
- ▶ matrices are 2D arrays.

Examples:

```
import numpy as np
```

```
x = np.array([1, 2, 3])          # vector (shape: (3,))  
A = np.array([[1, 2],  
              [3, 4]])          # matrix (shape: (2, 2))
```

Important: Shape determines what operations are mathematically valid.

WHY SHAPES MATTER

NumPy enforces linear algebra rules through array shapes.

For example:

- ▶ matrix multiplication requires compatible dimensions,
- ▶ dot products require matching vector lengths.

Shape mismatches are not just programming errors — they are *mathematical errors*.

Key idea: NumPy forces you to respect the math.

DOT PRODUCTS

The dot product computes a weighted sum of vector components.

Example:

```
x = np.array([1, 2, 3])
```

```
y = np.array([4, 5, 6])
```

```
np.dot(x, y)      # 32
```

This operation appears everywhere:

- ▶ projections,
- ▶ energy calculations,
- ▶ similarity measures.

MATRIX MULTIPLICATION

Matrix multiplication represents linear transformations.

In NumPy:

```
C = A @ B  
# or  
C = np.matmul(A, B)
```

The @ operator encodes true matrix multiplication — not elementwise multiplication.

Key idea: NumPy expresses mathematical intent directly.

SOLVING SYSTEMS OF EQUATIONS

A core problem in science is solving:

$$Ax = b$$

This arises in:

- ▶ physics simulations,
- ▶ circuit analysis,
- ▶ regression models.

NumPy provides a direct and stable solver.

USING `NP.LINALG.SOLVE`

```
A = np.array([[3.0, 1.0],  
              [1.0, 2.0]])  
b = np.array([9.0, 8.0])  
  
x = np.linalg.solve(A, b)
```

Important:

- ▶ this avoids explicitly computing A^{-1} ,
- ▶ it uses stable numerical methods internally.

Rule of thumb: Solve systems, do not invert matrices.

LEAST SQUARES: WHEN EXACT SOLUTIONS DON'T EXIST

Real data is noisy.

Often:

$$Ax = b \quad \text{has no exact solution}$$

Instead, we find the solution that minimizes error.

This is the basis of:

- ▶ regression,
- ▶ curve fitting,
- ▶ inverse problems.

LEAST SQUARES IN NUMPY

```
x, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
```

Interpretation:

- ▶ x : best-fit solution,
- ▶ residuals: total error,
- ▶ rank: effective dimensionality.

Key idea: Most scientific problems are approximate, not exact.

FLOATING-POINT NUMBERS ARE APPROXIMATE

Computers cannot represent most real numbers exactly.

Example:

$$0.1 + 0.2 \neq 0.3$$

In Python:

```
0.1 + 0.2 == 0.3    # False
```

This is not a bug — it is a fundamental limitation of floating-point arithmetic.

COMPARING FLOATING-POINT NUMBERS SAFELY

Instead of exact equality, use tolerance-based comparison:

```
np.isclose(a, b)
```

Key idea:

Numerical equality means “close enough,” not identical.

NUMERICAL DIFFERENTIATION

Computers approximate derivatives using finite differences.

Given sampled values of $f(x)$:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

NumPy makes this straightforward.

FINITE DIFFERENCES IN NUMPY

```
dx = x[1] - x[0]  
dfdx = np.diff(fx) / dx
```

Accuracy improves as the grid becomes finer, but numerical error never disappears completely.

NUMERICAL INTEGRATION

Integrals are approximated using sums.

A common method is the trapezoidal rule.

NumPy provides this directly.

TRAPEZOIDAL INTEGRATION

```
np.trapz(y, x)
```

Key idea: Accuracy depends on resolution and smoothness of the function.

EIGENVALUES AND EIGENVECTORS

Eigenvalues reveal intrinsic structure of linear systems.

```
vals, vecs = np.linalg.eig(A)
```

They appear in:

- ▶ stability analysis,
- ▶ vibrations,
- ▶ dimensionality reduction.

WHY THIS LEADS NATURALLY TO SCIPY

NumPy gives us:

- ▶ efficient arrays,
- ▶ linear algebra primitives,
- ▶ basic numerical tools.

SciPy builds on NumPy to provide:

- ▶ optimization algorithms,
- ▶ differential equation solvers,
- ▶ interpolation and statistics.

Big picture:

NumPy represents problems. SciPy solves them.

Part II

INTRODUCTION TO SCIPY

FROM NUMPY TO SCIPY

NumPy gives us:

- ▶ efficient numerical arrays,
- ▶ fast linear algebra primitives,
- ▶ basic numerical operations.

But many scientific problems require *algorithms*, not just array operations.

SciPy exists to answer the question:

How do we actually solve scientific problems using these arrays?

WHAT SciPy IS (AND IS NOT)

SciPy is a library built **on top of NumPy**.

SciPy is:

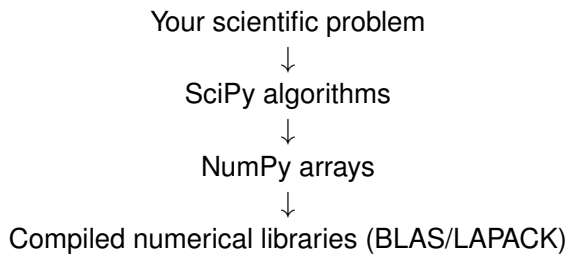
- ▶ a collection of well-tested numerical algorithms,
- ▶ focused on scientific and engineering problems,
- ▶ designed to work seamlessly with NumPy arrays.

SciPy is not:

- ▶ a replacement for NumPy,
- ▶ a machine learning library,
- ▶ a plotting library.

HOW NUMPY AND SCIPY FIT TOGETHER

Think of the scientific Python stack as layered:



Key idea:

NumPy represents data. SciPy operates on it.

CORE PROBLEM CLASSES IN SciPy

SciPy focuses on *numerical problem-solving*.

Examples include:

- ▶ optimization (finding minima and maxima),
- ▶ root finding ($f(x) = 0$),
- ▶ numerical integration and differentiation,
- ▶ solving differential equations,
- ▶ interpolation and curve fitting.

These problems arise everywhere in science and engineering.

MOTIVATING EXAMPLE: ROOT FINDING

Suppose we want to solve:

$$f(x) = x^3 - x - 2 = 0$$

This equation:

- ▶ has no closed-form solution,
- ▶ cannot be solved exactly by algebra,
- ▶ must be solved numerically.

This is a classic SciPy problem.

ROOT FINDING WITH SciPy

```
from scipy import optimize

def f(x):
    return x**3 - x - 2

root = optimize.root_scalar(f, bracket=[1, 2])
root.root
```

What SciPy is doing for us:

- ▶ choosing a stable numerical method,
- ▶ iterating until convergence,
- ▶ handling stopping criteria and edge cases.

MOTIVATING EXAMPLE: NUMERICAL INTEGRATION

Suppose we want:

$$\int_0^1 e^{-x^2} dx$$

This integral:

- ▶ has no elementary closed-form expression,
- ▶ appears in probability and physics,
- ▶ must be computed numerically.

SciPy provides adaptive integration algorithms.

INTEGRATION WITH SciPy

```
from scipy import integrate
import numpy as np

f = lambda x: np.exp(-x**2)
value, error = integrate.quad(f, 0, 1)
```

Key idea: SciPy automatically adapts the grid to achieve accuracy.

WHAT SCI-PY GIVES YOU CONCEPTUALLY

SciPy does not just compute numbers.

It encodes:

- ▶ numerical analysis theory,
- ▶ stability considerations,
- ▶ convergence guarantees,
- ▶ decades of scientific best practices.

Key idea:

SciPy lets you use expert-level algorithms without reimplementing them.

HOW SciPy FITS INTO THIS COURSE

In this course, we will use SciPy to:

- ▶ solve equations and optimization problems,
- ▶ integrate and differentiate numerically,
- ▶ solve ordinary differential equations,
- ▶ connect mathematical models to real data.

You will focus on:

- ▶ understanding the problem being solved,
- ▶ choosing the right SciPy tool,
- ▶ interpreting the results.

NEXT LECTURE: A DEEPER DIVE INTO SciPy

Next time, we will:

- ▶ explore SciPy submodules in detail,
- ▶ solve optimization problems,
- ▶ work with differential equations,
- ▶ compare numerical methods and accuracy.

Goal:

Learn to translate mathematical problems into SciPy code confidently.